

ANONYMUS

CONFIDENTIAL

Architectural & Security Specification

A Unified, Metadata-Resistant End-to-End Encrypted Messaging Application with Stateless Relay and P2P Onion Routing



Author: Core Architecture Team

Version: 1.2.0 (Unified)

Status: Release Candidate

Date: June 2026

Table of Contents

1. Executive Summary & Core Vision	2
2. System Architecture & Dual Transport Modes	3
3. Detailed Technical Stack	4
4. Cryptographic Specifications	5
5. Security Hardening & Client Protections	6
6. Enterprise Use-Cases & Customer Value	7
7. Methodology & Future Development Roadmap	8

1. Executive Summary & Core Vision

In the modern digital landscape, communication channels are subject to pervasive surveillance, active interception, and systematic metadata harvesting. Conventional secure messengers implement end-to-end encryption (E2EE) but fail to address metadata exposure, leaving user identities, IP addresses, communication frequencies, and social graphs vulnerable to passive network observers.

AnonyMus addresses these vulnerabilities through a paradigm-shifting, zero-knowledge, unified communication architecture. It bridges two distinct network topologies—stateless, centralized relay messaging and fully decentralized, peer-to-peer onion routing—into a single executable.

The project's primary vision is to deliver **absolute confidentiality** and **metadata resistance**. By decoupling the presentation layer from the underlying transport protocol, AnonyMus allows operators and clients to dynamically hot-swap their network topologies at runtime based on their threat model, physical location, and infrastructure constraints.

Core Architecture Philosophy

- 1. Zero Leakage by Default:** Data is encrypted before leaving RAM; metadata is actively obfuscated or routed through Tor.
- 2. Zero Server Knowledge:** Relay servers are stateless conduits; they store no history, user directories, or public key rings.
- 3. Runtime Elasticity:** Clients can pivot from high-speed relay pipelines to metadata-anonymous Tor tunnels seamlessly.

2. System Architecture & Dual Transport Modes

Anonymus unifies two transport architectures within a single codebase, coordinated by a root WSGI dispatcher.

A. Centralized Relay Mode

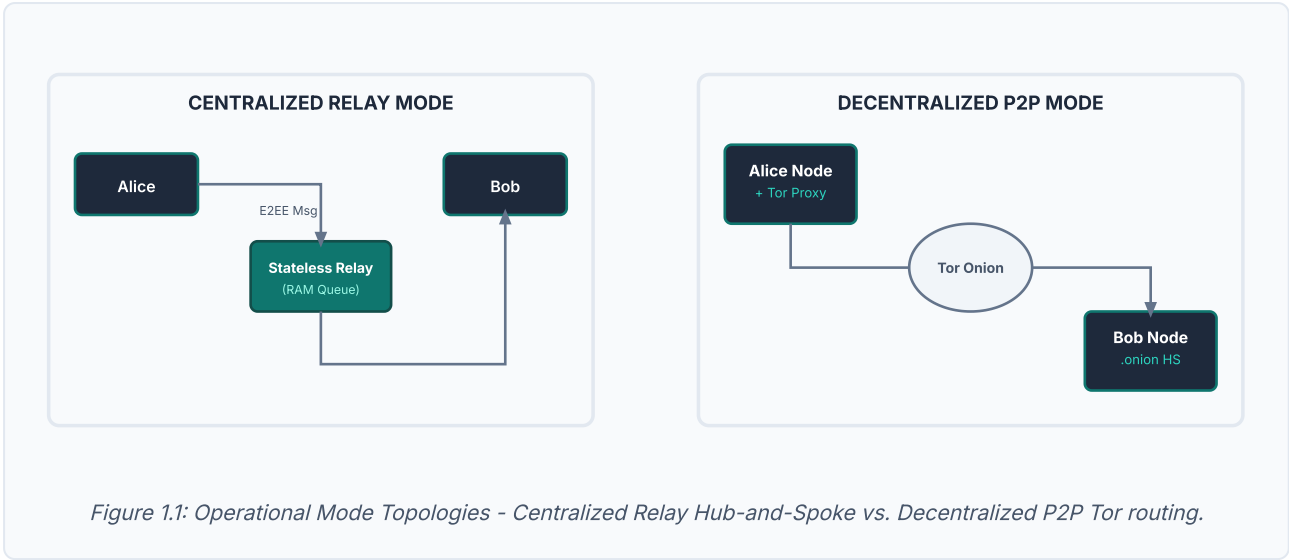
In centralized relay mode, the system operates in a hub-and-spoke layout. Communication between clients is brokered by a stateless Flask server utilizing WebSocket connections (via Flask-SocketIO) for low-latency, real-time message delivery.

- **Stateless Memory Queues:** The relay server maintains message queues strictly in volatile memory. Messages are removed immediately upon confirmation of receipt, preventing the accumulation of forensic evidence.
- **Queue Authorization:** Secure UUIDs represent active communication channels. To prevent traffic analysis and unauthorized ingestion, WebSocket clients must prove channel ownership using cryptographic session tokens.
- **Local Area mDNS Discovery:** Servers advertise availability over multicast DNS (using standard `_anonymus._tcp.local.` records), enabling clients to automatically locate and bind to a local server relay without exposing external network configurations.

B. Decentralized Peer-to-Peer (P2P) Mode

In decentralized mode, the centralized relay server is bypassed entirely. Every client node operates its own local Flask microservice bound to localhost and runs an integrated Tor client daemon.

- **Embedded Tor Hidden Services:** The transport manager orchestrates a local Tor Expert Bundle, generating an ephemeral `.onion` hidden service. NAT traversal, firewall penetration, and direct peer routing are achieved natively.
- **Tor SOCKS5 Outbound Proxying:** Outbound requests are routed through Tor SOCKS5 (`127.0.0.1:9050`), preventing peers from disclosing their true IP address or network coordinates.
- **AES-256-GCM Local Database Encryption:** The node databases (`local_node.db`), storing contact lists, identity keys, and message history, are encrypted at rest using AES-256-GCM, with keys derived from user passwords using PBKDF2.



3. Detailed Technical Stack

Anonymus utilizes a carefully curated stack to guarantee low-level hardware control, memory safety, cross-platform compatibility, and robust cryptographic isolation.

Layer	Centralized Relay Mode	Decentralized P2P Mode
Core Language	Python 3.11+, JavaScript (ES6)	Python 3.11+, Kotlin (Android)
Web Framework	Flask 3.0.3, WSGI Dispatcher	Flask 3.0.3 (localhost-bound API)
Networking / Comm	Flask-SocketIO, Eventlet, mDNS	Tor SOCKS5, PySocks, requests[socks]
Database	PostgreSQL 15 / SQLite (auth only)	SQLite (local, fully encrypted at rest)
Cryptographic Engine	PyCa/cryptography, WebCrypto API	PyCa/cryptography, Google Tink (Android)
Launcher / Packaging	Docker, Docker Compose	Tkinter, PyInstaller, Inno Setup

A. Desktop Launcher & Administrative Camouflage

To resist detection by static port scanners, firewalls, and local administrative scans, the desktop deployment uses a graphical launcher constructed in Tkinter.

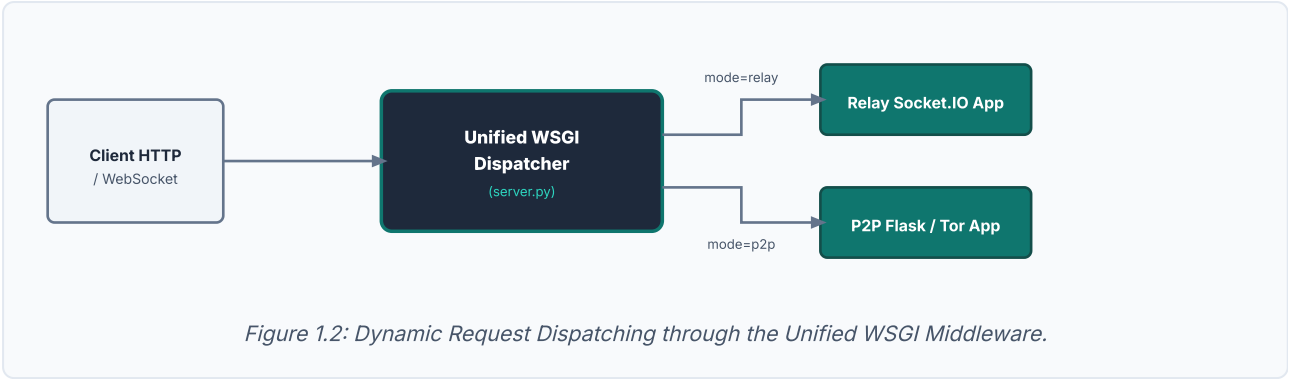
- Diagnostic Camouflage:** The GUI is themed as the "Windows Network Diagnostics & Adapter Utility," rendering authentic network connection information. It runs administrative processes in the background to handle Tor daemon initiation.
- Dynamic Port Allocation:** If the default port is blocked, the launcher binds the web frontend and proxy connections to dynamic random ports above 49152, evading signature-based local traffic blockers.

B. Native Android Implementation Stack

The Android client is designed as a standalone, native application.

- Jetpack Compose:** Utilizes modern, declarative UI layouts ensuring high performance and defensive UI rendering (such as automatic composition purging on app pause).
- Google Tink Cryptographic Engine:** Key isolation is achieved via Google Tink. Cryptographic key material is bound to the Android Keystore system, isolating keys from the general user-space memory of the OS.
- Trust-on-First-Use (TOFU) Certificate Pinning:** To prevent Man-in-the-Middle (MitM) attacks by enterprise root certificates or state actors, the client pins the relay server's TLS certificate on the

first connection, blocking future handshakes if the server cert changes.



4. Cryptographic Specifications

AnonyMus does not rely on third-party security parameters. Both modes of operation implement a customized cryptosystem providing end-to-end encryption (E2EE), forward secrecy, and metadata resistance.

A. Key Exchange & Session Setup

To initiate secure sessions, clients generate ephemeral Elliptic Curve Diffie-Hellman (ECDH) keypairs using the ****NIST P-256 (secp256r1)**** curve. The public keys are exchanged out-of-band (via QR code scanning or secure Onion exchange URLs). A master shared secret is generated via ECDH:

```
Master_Secret = ECDH_Shared_Secret(Client_Private, Peer_Public)
```

This master secret is immediately processed through ****HKDF-SHA256**** (with a static system salt and context info) to derive initial symmetric keys:

```
Key_Material = HKDF(Master_Secret, Salt, Context_Info, 92 bytes)
```

This key material is partitioned into dedicated encryption, decryption, and authentication keys.

B. Message Forward Secrecy (Symmetric HKDF Ratchet)

To prevent exposure of historical conversations in the event of an identity key compromise, AnonyMus implements a unidirectional key ratchet. The active chain key is advanced for every message sent or received:

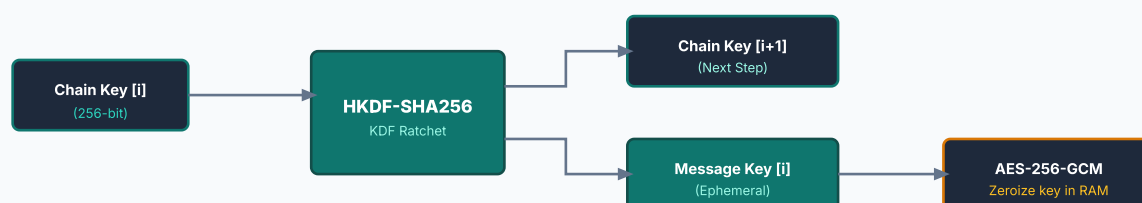


Figure 1.3: Ephemeral Key Generation via Unidirectional Symmetric Ratchet.

The message key is ephemeral and immediately zeroed out in memory (overwritten with zeroes in RAM) once the message payload is encrypted or decrypted. This limits exposure of keys in case of physical device capture.

C. Authenticated Encryption with Associated Data (AEAD)

Payload encryption utilizes **AES-256-GCM** with a cryptographically secure random 12-byte Initialization Vector (IV). To bind ciphertext securely to its metadata, sorted cryptographic fingerprints of the participants (derived from identity keys, known as Safety Numbers) and the system version are injected as **Associated Data (AAD)**. This prevents payload injection, replay attacks, or conversation context manipulation.

5. Security Hardening & Client Protections

Security is implemented depth-first across the user-facing web applications and Android devices.

A. Web Client Memory Safety & Content Delivery

- **Chunked Base64 Conversion:** To protect browser execution stacks when handling massive media payloads, Anonymus utilizes a customized, recursive chunked array buffer conversion library, avoiding memory allocation spikes and browser crashes.
- **Disappearing Messages:** Communication deletion timers are negotiated out-of-band using control frames. Once timers expire, the client-side JavaScript removes the message DOM node, deletes key parameters, and forces garbage collection.
- **Log Sanitation Engine:** To prevent exposure during forensic collection, all runtime console outputs and server log files are passed through a regex-based filter that redacts Base64 key material, authorization headers, IP addresses, and session UUIDs.

B. Native Android Hardening Mechanisms

- **Biometric Security Lockout:** Access to the local client databases and UI views is controlled via Android's `BiometricPrompt` framework, integrating fingerprint or face biometric authentication with fallback to secure hardware device PINs.
- **FLAG_SECURE Enforcement:** The Android application layout invokes the OS-level `WindowManager.LayoutParams.FLAG_SECURE` . This prevents the operating system from capturing screenshots, blocks background screen sharing, and hides app views in the "Recent Apps" launcher list.
- **Sandboxed Database:** Database components are stored within private application containers (`/data/data/ ...`) with file read/write permissions limited exclusively to the application UID, preventing inter-process inspection.

Threat Model & Security Guarantees

Threat Vector	Anonymus Mitigation Strategy
Active Server Compromise	E2EE prevents message interception. Relay stores no keys or history.
ISP / Network Surveillance	Tor hidden routing conceals physical IP coordinates.
Physical Device Capture	AES-256 encrypted database; keys stored securely in Keystore.

6. Enterprise Use-Cases & Customer Value

AnonyMus offers extreme value in environments requiring high secrecy, low infrastructure overhead, and robust metadata resistance.

1. Government & Diplomatic Missions

Diplomatic envoys operating in hostile jurisdictions require communication channels that hide not just the message content, but their very communication network. Using P2P mode over Tor hidden services, diplomats can communicate directly with HQ, circumventing host-nation ISP surveillance, packet inspection, and localized network blocks.

2. Financial Services & M&A Negotiations

During high-value mergers and acquisitions (M&A) or corporate strategy developments, communication leaks can lead to insider trading and millions of dollars in damages. The centralized relay mode, deployed as an internal stateless container stack (PostgreSQL + Redis + Flask), guarantees that negotiation channels leave no audit trail or data residues on servers, minimizing liability.

3. Secure Corporate Whistleblowing

Corporate compliance frameworks require secure, anonymous reporting tools. The P2P mode serves as a zero-knowledge whistleblower drop box. A whistleblowing system hosted as a `.onion` service allows employees to report misconduct without generating network logs on corporate proxies or Active Directory servers, protecting their identity.

4. Critical Infrastructure Operations

SCADA networks, utilities, and security control rooms must exchange critical diagnostic and command messages over air-gapped or localized networks. Using the local area mDNS discovery feature in centralized relay mode, operators can spin up an ad-hoc local server relay and begin encrypted chat communications instantly over local Wi-Fi or ethernet networks, without requiring internet connectivity or DNS setup.

5. Investigative Journalism & Human Rights

Journalists in high-risk zones rely on secure channels to interact with sensitive sources. The Android client's anti-screenshot flags, combined with P2P metadata resistance over Tor, protect sources from physical and network-level identification, preventing leakage of draft stories or source identity records.

7. Methodology & Future Development Roadmap

AnonyMus follows rigorous software engineering methodologies to maintain codebase reliability, portability, and testing verification.

A. Unified Codebase & Dynamic Mode Swapping

Instead of maintaining two separate codebases (which increases maintenance costs and security desynchronization), the P2P and Centralized Relay transport modules are unified. The application maintains clear interfaces:

```
class TransportProvider(ABC):
    def start(self, config: Dict[str, Any]) → None: ...
    def stop(self) → None: ...
    def send(self, recipient: str, ciphertext: str, iv: str, seq: int) → bool: ...
    def handoff(self, other: TransportProvider) → None: ...
    def health(self) → Dict[str, Any]: ...
```

Mode switches trigger a session handoff, halting active connections, zeroing volatile RAM session keys, re-binding transport sockets, and instantiating the target protocol.

B. Verification Plan & Testing

The system includes a suite of automated unit and integration tests located in `tests/`. This includes testing cryptographic key ratcheting, dynamic WSGI routing, mDNS discovery, and P2P SQLite database encryption.

C. Future Development Roadmap

- **Post-Quantum Cryptography (PQC) Integration:** Upgrading the P-256 ECDH exchange with hybrid post-quantum key encapsulation mechanisms (such as ML-KEM / Kyber), preventing future decryption of recorded traffic by quantum computers.
- **Metadata Obfuscation (Chaff Traffic):** Injecting decoy messages and dummy network traffic at randomized intervals in centralized relay mode to mask user conversation metadata from deep network analysis.
- **Federated Relay Networks:** Enabling individual communities to host independent relay servers that federate traffic securely, eliminating reliance on a single central relay server infrastructure.
- **Multi-Device Sync protocol:** Developing an E2EE session state synchronizer utilizing secure QR-code scanning to securely replicate ratchet states across desktop and mobile clients.